



University
Mohammed VI
Polytechnic



UNIVERSITY MOHAMMED VI POLYTECHNIC
College of Computing

INTERNSHIP REPORT

Carried out at

Administration of Customs and Indirect Taxes
(Administration des Douanes et Impôts Indirects, Rabat)

Predicting Market Prices from E-Commerce Metadata

A Feasibility Study for Customs Risk Assessment

*An end-to-end pipeline combining web scraping,
LLM-based feature extraction, and regression modelling*

Submitted by:

Zaina Aitahmadoubrahim

Supervisor:

Hind Lamharhar

Field of Study:

Computer Science

Host Organisation:

ADII, Morocco

Internship Period:

June 2026 – July 2026

Academic Year:

2025 – 2026

Remerciements

Je tiens à exprimer ma profonde gratitude envers toutes les personnes qui ont contribué, de près ou de loin, au bon déroulement de ce stage.

Mes remerciements les plus sincères s'adressent en premier lieu à ma superviseuse de stage, **Mme Hind Lamhaehar**, pour sa disponibilité, ses conseils éclairés et son accompagnement précieux tout au long de ce travail. Ses orientations ont été déterminantes dans la définition de mes missions et dans la rigueur de leur exécution.

Je tiens également à remercier chaleureusement **M. Mohammed Afoulki** pour son encadrement attentif, son soutien technique constant et sa grande disponibilité au quotidien. Ses conseils avisés et son partage d'expertise ont grandement facilité la mise en œuvre pratique de mes tâches.

Je remercie également **l'Administration des Douanes et Impôts Indirects (ADII)** pour m'avoir accueillie au sein de ses équipes et offert un cadre de travail stimulant, propice à l'apprentissage et à l'innovation. Cette expérience m'a permis de mesurer concrètement la valeur des outils d'intelligence artificielle dans un contexte institutionnel et opérationnel exigeant.

Mes remerciements vont également à **l'Université Mohammed VI Polytechnique** et au **College of Computing** pour la formation solide qu'ils m'ont dispensée, et qui a constitué le socle nécessaire à la réalisation de ce stage.

Zaina Aitahmadoubrahim

Juillet 2026

Abstract

This report presents the work carried out during a one-month internship at the Administration of Customs and Indirect Taxes (ADII), Morocco. The project addresses a concrete operational need: building an automated system capable of estimating the fair market retail price of products from publicly available product descriptions, with direct relevance to customs valuation procedures.

The pipeline encompasses four main phases. First, a compliant web scraper was developed using Selenium to collect a dataset of over 10,000 product listings across nine categories from Jumia Morocco, the leading e-commerce platform in the country. Second, the raw dataset was cleaned and enriched through LLM-based Information Extraction using the Mistral AI API, which intelligently decomposes unstructured product titles into three structured semantic fields: *name*, *usage*, and *composition*. Third, two distinct feature engineering strategies were designed and compared: (i) TF-IDF vectorisation combined with One-Hot Encoding, and (ii) dense semantic embeddings produced by the BGE-M3 multilingual model combined with Mean Target Encoding. Finally, two regression models were trained and evaluated: a Random Forest Regressor and a tuned XGBoost Regressor optimised through Randomised Search and 3-fold cross-validation.

Both approaches achieved comparable results, with an R^2 score of approximately 0.80 and a Mean Absolute Error of approximately 155 MAD, demonstrating that product information available at listing time carries strong predictive signals for price estimation.

Keywords: Machine Learning, Price Prediction, Web Scraping, TF-IDF, Sentence Embeddings, BGE-M3, Random Forest, XGBoost, NLP, E-commerce, Jumia Morocco, Customs Valuation.

Contents

Remerciements	1
Abstract	2
1 Introduction	6
1.1 Host Organisation: ADII	6
1.2 Project Objectives	7
1.3 Report Structure	7
2 Data Collection and Preprocessing	8
2.1 Target Platform: Jumia Morocco	8
2.2 Compliance and Ethical Considerations	8
2.3 Web Scraping Pipeline	9
2.3.1 Category Coverage	9
2.4 Data Cleaning	10
2.5 Log-Price Transformation	10
2.6 LLM-Based Information Extraction	11
3 First Approach: TF-IDF, One-Hot Encoding, and Random Forest	12
3.1 Overview	12
3.2 Feature Engineering	12
3.2.1 Categorical Features: One-Hot Encoding	12
3.2.2 Textual Features: TF-IDF	13
3.2.3 Combined Feature Matrix	13
3.3 Why Not Linear Regression?	14
3.4 Random Forest Regressor	14
3.4.1 Algorithm and Justification	14
3.4.2 Training and Evaluation	15

3.5	Results	15
4	Second Approach: BGE-M3, Target Encoding, and XGBoost	17
4.1	Overview and Motivation	17
4.2	LLM-Based Feature Extraction	17
4.3	Feature Engineering	18
4.3.1	Categorical Features: Mean Target Encoding	18
4.3.2	Textual Features: BGE-M3 Embeddings	18
4.3.3	Textual Features: BGE-M3 Embeddings	18
4.3.4	Combined Feature Matrix	19
4.4	XGBoost Regressor	20
4.4.1	Algorithm	20
4.4.2	Hyperparameter Tuning	20
4.4.3	Training	20
4.5	Results	21
5	Comparative Analysis and Discussion	22
5.1	Performance Comparison	22
5.2	Discussion	22
5.2.1	Why Performance Did Not Improve in the Second Approach . .	23
5.2.2	Significance of $R^2 = 0.80$ in the Customs Context	23
5.3	Limitations	23
6	Conclusion and Future Work	24
6.1	Conclusion	24
6.2	Future Work	25

List of Tables

2.1	Raw features extracted from Jumia product listings	9
2.2	Dataset distribution by main category	10
3.1	Effect of the brand frequency threshold	13
3.2	Feature matrix dimensions — First approach	14
3.3	Random Forest evaluation results	15
4.1	Feature matrix dimensions — Second approach	19
4.2	Hyperparameter search space and optimal values found	20
4.3	XGBoost evaluation results — Second approach	21
5.1	Side-by-side comparison of both approaches	22

Chapter 1

Introduction

1.1 Host Organisation: ADII

The **Administration des Douanes et Impôts Indirects (ADII)** is the Moroccan governmental authority responsible for regulating, controlling, and taxing goods crossing the national borders. Among its main missions, ADII ensures the correct application of customs duties, facilitates trade compliance, and combats fraudulent practices, including the undervaluation of imported goods.

A key concept in this context is the *valeur en douane* (customs value), which represents the taxable base used for calculating import duties. According to the WTO Agreement on Customs Valuation, this value is generally based on the transaction value declared by the importer. However, when there are doubts regarding the accuracy of a declared value, such as when a declared price appears significantly lower than expected, customs authorities require reliable reference information to assess the plausibility of the declaration.

This project explores the use of Machine Learning techniques to develop a data-driven price reference system. By using product metadata such as name, brand, category, and technical specifications, the developed model estimates the expected retail price of consumer products available on Jumia Morocco, one of the major e-commerce platforms in the country.

Although online retail prices do not represent official customs values, they provide

valuable market information that can serve as a reference indicator. Therefore, such a system could potentially support customs officers by providing an additional benchmark when evaluating the consistency of declared values for imported consumer goods.

1.2 Project Objectives

The primary objective of this internship project is to design and implement a complete machine learning pipeline for product price prediction on Jumia Morocco. Specifically, the project aims to:

- Collect a large, representative and compliant dataset of Jumia Morocco product listings through automated web scraping.
- Clean, preprocess, and enrich the collected data using modern Natural Language Processing (NLP) techniques, including LLM-based information extraction.
- Design and compare two distinct feature engineering strategies of increasing sophistication.
- Train, tune, and evaluate multiple regression models and compare their performance.
- Establish a proof-of-concept price estimation tool applicable to the customs valuation context.

1.3 Report Structure

Chapter 2 describes the data collection and preprocessing pipeline, including the web scraping architecture, data cleaning steps, and LLM-based feature extraction. Chapter 3 presents the first experimental approach (TF-IDF + One-Hot Encoding + Random Forest). Chapter 4 presents the second, more advanced approach (BGE-M3 embeddings + Target Encoding + XGBoost). Chapter 5 provides a comparative analysis and discussion of both approaches. Chapter 6 concludes the report and outlines future directions.

Chapter 2

Data Collection and Preprocessing

2.1 Target Platform: Jumia Morocco

Jumia Morocco (<https://www.jumia.ma>) is one of the largest e-commerce marketplaces in Morocco, offering a wide range of consumer products across various categories, including electronics, home appliances, fashion, beauty, and sporting goods. The platform hosts products from numerous independent sellers, providing broad coverage of Moroccan retail market prices. This makes it a valuable data source for studying product price patterns and developing a reference pricing system that can support applications related to market analysis and customs valuation.

2.2 Compliance and Ethical Considerations

Before initiating data collection, the platform's `robots.txt` file was carefully reviewed. Jumia Morocco explicitly *permits* automated scraping under the following conditions:

- The bot must identify itself with an honest `User-Agent` string.
- The request rate must not exceed 200 requests per minute.
- IP rotation to circumvent this limit is explicitly prohibited.

All scraping activities in this project were conducted in strict compliance with these directives.

2.3 Web Scraping Pipeline

The data collection pipeline was implemented using **Selenium** with the Microsoft Edge browser driver, which enables the rendering of JavaScript-generated content not accessible via simple HTTP requests. The scraper navigates category listing pages, waits for dynamic content to load, and extracts product information from the rendered HTML.

For each product card, attributes were extracted from the `data-ga4-*` HTML attributes embedded by Jumia’s analytics layer:

Table 2.1 – Raw features extracted from Jumia product listings

Field	HTML Source	Description
<code>name</code>	<code>data-ga4-item_name</code>	Full product title as listed
<code>brand</code>	<code>data-ga4-item_brand</code>	Product brand
<code>category</code>	Page context	Main product category
<code>price</code>	<code>.prc</code> CSS class	Listed price in MAD

Products with price ranges (e.g., *73.47 – 185.07 Dhs*) were handled by retaining the lower bound. The scraper included automatic retry logic and incremental saving after each page to ensure robustness against connection failures.

2.3.1 Category Coverage

The scraper was deployed across ten main categories and numerous subcategories, yielding the following distribution:

Table 2.2 – Dataset distribution by main category

Category	Products	Share
Sport & Loisirs	2,357	23.6%
Téléphone & Tablette	1,844	18.5%
Jeux Vidéos	1,760	17.6%
Beauté & Santé	1,268	12.7%
Informatique	879	8.8%
Mode	768	7.7%
Accessoires Auto	546	5.5%
Electroménager	401	4.0%
Maison & Bureau	170	1.7%
Total	9,993	100%

2.4 Data Cleaning

The raw dataset underwent the following cleaning steps:

1. **Brand standardisation:** brand names were normalised to title case (e.g., SAMSUNG $\rightarrow$ Samsung).
2. **Outlier removal:** for each main category, products with prices below 10 MAD or exceeding the category mean by more than three standard deviations were discarded as likely data errors.
3. **Duplicate removal:** product entries with identical name and price combinations were eliminated.
4. **Feature selection:** the `discount` column was dropped following the project supervisor’s recommendation, and the `sub_category` column was removed as it was near-identical to `main_category` for the majority of rows.

2.5 Log-Price Transformation

Product prices span a wide range. From under 30 MAD for small accessories to over 16,000 MAD for premium smartphones. This right-skewed distribution causes regression models to treat absolute prediction errors inconsistently: a 500 MAD error is catastrophic for a 200 MAD product but negligible for a 15,000 MAD one.

To address this, the transformation $\log_price = \log(1 + p)$ was applied to the price p . This yields a more symmetric distribution and ensures the model treats errors

proportionally across all price ranges. Predictions are converted back to MAD at evaluation time using the inverse $\hat{p} = \exp(\hat{y}) - 1$.

2.6 LLM-Based Information Extraction

Raw Jumia product titles are dense, unstructured strings concatenating multiple attributes without clear delimiters. For example:

“Redmi A7 Pro 4G NFC 6.9” – 128Gb - 4GB up to 8GB RAM - 6000mAH - Black”

Using the **Mistral AI API** (`mistral-small-latest`), each title was automatically decomposed into three structured semantic fields:

- **name:** short clean product identifier, e.g. *Redmi A7 Pro*.
- **usage:** primary use in one sentence, e.g. *Smartphone for communication and multimedia*.
- **composition:** key technical attributes, e.g. *4G NFC, 6.9 inch, 128GB, 4GB RAM, 6000mAh, Black*.

Products were processed in batches of 10 to minimise API latency, with automatic retry logic and incremental saving every 100 rows to ensure robustness against connection interruptions. This LLM-based decomposition is used exclusively in the second experimental approach (Chapter 4); the first approach operates on the raw **name** column.

Chapter 3

First Approach: TF-IDF, One-Hot Encoding, and Random Forest

3.1 Overview

The first experimental approach establishes a solid baseline using classical text and categorical encoding techniques, trained with a Random Forest Regressor. It serves both as a performance benchmark and as a foundation for understanding the limitations that motivated the second approach.

3.2 Feature Engineering

3.2.1 Categorical Features: One-Hot Encoding

The `brand` and `main_category` columns were encoded using One-Hot Encoding. The dataset contains 838 unique brands; directly encoding all of them would introduce 838 binary columns, leading to the *curse of dimensionality*. To mitigate this, only brands appearing in at least 20 products were retained; the rest were grouped as `Other`:

Table 3.1 – Effect of the brand frequency threshold

	Count	Coverage
Brands with ≥ 20 products (retained)	62	76.3%
Brands with < 20 products (grouped as <code>Other</code>)	776	23.7%
Total	838	100%

This yields 63 columns for brand and 9 for `main_category`, giving **72 categorical columns**.

3.2.2 Textual Features: TF-IDF

TF-IDF (Term Frequency – Inverse Document Frequency) converts text columns into numerical feature vectors by assigning each word a score reflecting its importance in a specific document relative to the entire corpus:

$$\begin{aligned}
 \text{TF}(w, d) &= \frac{\text{count of } w \text{ in } d}{\text{total words in } d} \\
 \text{IDF}(w) &= \log\left(\frac{N}{|\{d : w \in d\}|}\right) \\
 \text{TF-IDF}(w, d) &= \text{TF}(w, d) \times \text{IDF}(w)
 \end{aligned}$$

The `product_name` and `product_details` columns (obtained from the first round of LLM extraction, which split the raw title into a short name and a details string) were each vectorised retaining the top 100 most informative terms (`max_features=100`). This removes both very common function words (e.g. *de*, *le*) and extremely rare terms. Both vectorisers were fitted exclusively on the training set and then applied to the test set.

3.2.3 Combined Feature Matrix

The three components were horizontally concatenated into a single sparse matrix:

$$\mathbf{X}_{\text{final}} = \left[\mathbf{X}_{\text{OHE}} \parallel \mathbf{X}_{\text{TF-IDF name}} \parallel \mathbf{X}_{\text{TF-IDF details}} \right] \quad (3.1)$$

Table 3.2 – Feature matrix dimensions — First approach

Component	Columns	Type
One-Hot Encoding (brand + category)	72	Sparse binary
TF-IDF (<code>product_name</code>)	100	Sparse float
TF-IDF (<code>product_details</code>)	100	Sparse float
Total	272	Sparse

After transforming the raw product information into numerical features, the next step is to select an appropriate regression model. Since the objective is to predict product prices, several regression algorithms can be considered. However, due to the characteristics of the dataset, particularly the non-linear relationship between product attributes and prices, linear regression was not considered suitable.

3.3 Why Not Linear Regression?

Linear regression requires a linear relationship between each feature and the target. To assess this empirically, the price distribution was visualised across the top 15 brands. The box plot reveals abrupt, non-uniform jumps in median price: generic brands cluster around 100 MAD while Samsung products centre around 2,000 MAD. A straight line cannot represent such discontinuities. Linear regression was therefore ruled out in favour of a tree-based model.

3.4 Random Forest Regressor

3.4.1 Algorithm and Justification

A Random Forest constructs T decision trees, each trained on a bootstrap sample (random rows with replacement) of the training data, and using a random subset of features at each node split. The final prediction is the average over all trees:

$$\hat{y} = \frac{1}{T} \sum_{t=1}^T h_t(\mathbf{x}) \quad (3.2)$$

This double randomisation (rows and features) ensures diversity among trees, reducing variance without significantly increasing bias. Key advantages for this dataset include: natural handling of non-linear relationships, no requirement for feature scaling, and built-in regularisation through averaging.

3.4.2 Training and Evaluation

```

1 from sklearn.ensemble import RandomForestRegressor
2 import numpy as np
3 from sklearn.metrics import mean_squared_error,
   mean_absolute_error, r2_score
4
5 model = RandomForestRegressor(n_estimators=100, random_state
   =42)
6 model.fit(X_train, y_train)
7
8 # Convert predictions back from log-scale to MAD
9 y_pred_price = np.expml(model.predict(X_test))
10 y_test_price = np.expml(y_test)
11
12 rmse = np.sqrt(mean_squared_error(y_test_price, y_pred_price))
13 mae = mean_absolute_error(y_test_price, y_pred_price)
14 r2 = r2_score(y_test_price, y_pred_price)

```

Listing 3.1 – Random Forest training (*train_random_forest.py*)

The parameter `random_state=42` ensures fully reproducible results across runs.

3.5 Results

Table 3.3 – Random Forest evaluation results

Metric	Value
RMSE	388.77 MAD
MAE	155.88 MAD
R^2	0.8035

Interpretation:

- **MAE = 155.88 MAD:** on average, the predicted price deviates from the true price by approximately 156 MAD.
- **RMSE = 388.77 MAD:** the gap between RMSE and MAE indicates the presence of occasional large prediction errors, consistent with the underrepresentation of certain categories (notably *Maison & Bureau* with only 170 training examples).

- $R^2 = 0.8035$: the model explains approximately 80% of price variance from product metadata alone, a strong result given the highly heterogeneous categories and price range (30 to 16,000 MAD).

Chapter 4

Second Approach: BGE-M3, Target Encoding, and XGBoost

4.1 Overview and Motivation

The first approach, while effective, has two notable limitations:

1. **TF-IDF is semantically blind.** Words with similar meanings but different spelling (e.g. *ordinateur portable* and *laptop*) are treated as unrelated features. TF-IDF captures word frequency, not meaning.
2. **One-Hot Encoding is uninformative about price.** Binary columns indicate brand membership but carry no information about the typical price level associated with that brand.

The second approach addresses both by: (i) using richer LLM-extracted features (three semantic fields instead of two), (ii) replacing TF-IDF with dense semantic embeddings (BGE-M3), and (iii) replacing One-Hot Encoding with Mean Target Encoding.

4.2 LLM-Based Feature Extraction

As described in Section 2.6, the Mistral AI API was used to decompose raw product titles into three structured fields: **name**, **usage**, and **composition**. This richer decomposition provides more semantically coherent text segments for the embedding model to encode.

4.3 Feature Engineering

4.3.1 Categorical Features: Mean Target Encoding

Mean Target Encoding replaces each categorical value with the mean of the target variable (`log_price`) across all training examples belonging to that category:

$$\text{enc}(c) = \frac{1}{|S_c^{\text{train}}|} \sum_{i \in S_c^{\text{train}}} \log(1 + p_i) \quad (4.1)$$

Data leakage prevention: means were computed exclusively from the training set. The resulting lookup table was then applied to both training and test sets identically. For brands or categories not seen during training, the global training mean was used as a fallback.

4.3.2 Textual Features: BGE-M3 Embeddings

BGE-M3 (BAAI General Embedding, version M3) is a state-of-the-art multilingual sentence embedding model that maps any text to a dense 1024-dimensional vector in a semantic space, where semantically similar texts lie close together:

$$\mathbf{e}_i = \text{BGE-M3}(\text{text}_i) \in \mathbb{R}^{1024} \quad (4.2)$$

4.3.3 Textual Features: BGE-M3 Embeddings

BGE-M3 (BAAI General Embedding, version M3) is a state-of-the-art multilingual sentence embedding model that maps any text to a dense 1024-dimensional vector in a semantic space, where semantically similar texts lie close together:

$$\mathbf{e}_i = \text{BGE-M3}(\text{text}_i) \in \mathbb{R}^{1024} \quad (4.3)$$

BGE-M3 was selected for three reasons:

- **Multilingual support:** the dataset contains product titles in both French and English; BGE-M3 handles both naturally.
- **Semantic richness:** unlike TF-IDF, BGE-M3 recognises that *ordinateur portable* and *laptop* describe the same product type.
- **State-of-the-art performance:** BGE-M3 consistently ranks among the top multilingual embedding models on public benchmarks.

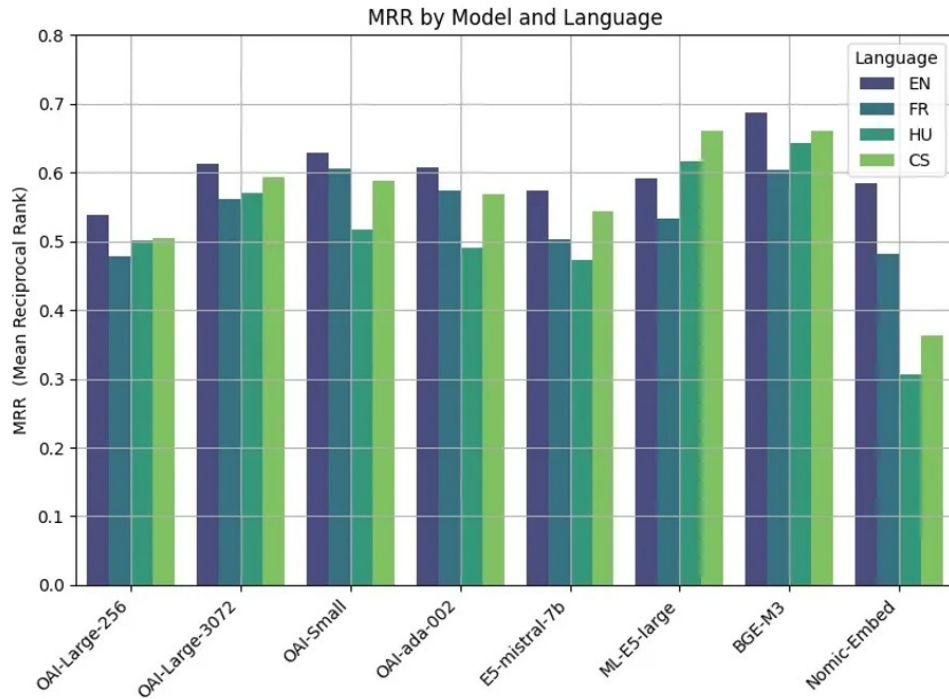


Figure 4.1 – Performance comparison of BGE-M3 against OpenAI and other leading multilingual embedding models across standard benchmarks. BGE-M3 achieves state-of-the-art results while remaining open-source and freely accessible.

Each of the three text columns (`name`, `usage`, `composition`) was embedded independently, with each embedding matrix saved to disk immediately upon completion to ensure robustness against computational interruptions (the full embedding process takes approximately one hour on CPU).

4.3.4 Combined Feature Matrix

$$\mathbf{X}_{\text{final}} = [\mathbf{E}_{\text{name}} \parallel \mathbf{E}_{\text{usage}} \parallel \mathbf{E}_{\text{composition}} \parallel \mathbf{x}_{\text{brand_enc}} \parallel \mathbf{x}_{\text{cat_enc}}] \quad (4.4)$$

Table 4.1 – Feature matrix dimensions — Second approach

Component	Columns	Type
BGE-M3 embedding (<code>name</code>)	1,024	Dense float
BGE-M3 embedding (<code>usage</code>)	1,024	Dense float
BGE-M3 embedding (<code>composition</code>)	1,024	Dense float
Target encoding (<code>brand</code>)	1	Dense float
Target encoding (<code>main_category</code>)	1	Dense float
Total	3,074	Dense

4.4 XGBoost Regressor

4.4.1 Algorithm

XGBoost implements gradient boosting: trees are built *sequentially*, each trained to predict the residuals of the cumulative model up to that point. Let $F_t(\mathbf{x})$ denote the model after t trees:

$$F_0(\mathbf{x}) = 0 \quad (4.5)$$

$$F_t(\mathbf{x}) = F_{t-1}(\mathbf{x}) + \eta \cdot h_t(\mathbf{x}) \quad (4.6)$$

where $h_t(\mathbf{x})$ is a base learner (decision tree) scaled by the learning rate $\eta \in (0, 1]$. At each step t , the new tree h_t is fit to minimize a regularized objective function based on the first and second-order gradients of the loss function evaluated at $F_{t-1}(\mathbf{x})$. In the specific case of a squared-error loss, this objective simplifies to fitting h_t to the residuals of F_{t-1} .

4.4.2 Hyperparameter Tuning

To find the optimal hyperparameters, **Randomised Search** was combined with **3-fold cross-validation**. Rather than exhaustively testing all $3^5 = 243$ combinations in the grid, 20 random combinations were sampled (`n_iter=20`), yielding $20 \times 3 = 60$ training runs in total.

Table 4.2 – Hyperparameter search space and optimal values found

Hyperparameter	Values Tested	Optimal Value
<code>n_estimators</code>	[200, 300, 500]	500
<code>learning_rate</code>	[0.01, 0.05, 0.1]	0.05
<code>max_depth</code>	[4, 6, 8]	8
<code>subsample</code>	[0.7, 0.8, 0.9]	0.9
<code>colsample_bytree</code>	[0.7, 0.8, 0.9]	0.9

The scoring metric was `neg_mean_absolute_error`: scikit-learn’s framework maximises the scoring function, so negating MAE turns minimisation into maximisation.

4.4.3 Training

```

1 from xgboost import XGBRegressor
2
3 model = XGBRegressor(
4     n_estimators=500, learning_rate=0.05,
5     max_depth=8, subsample=0.9,
6     colsample_bytree=0.9, random_state=42,
7     tree_method="hist" # histogram-based algorithm for CPU
8     efficiency
9 )
10 model.fit(X_train, y_train,
11           eval_set=[(X_test, y_test)], verbose=50)
12
13 y_pred_price = np.expml(model.predict(X_test))
14 y_test_price = np.expml(y_test)

```

Listing 4.1 – XGBoost training with tuned hyperparameters (`train_xgboost.py`)

4.5 Results

Table 4.3 – XGBoost evaluation results — Second approach

Metric	Value
RMSE	388.83 MAD
MAE	155.31 MAD
R^2	0.8034

Chapter 5

Comparative Analysis and Discussion

5.1 Performance Comparison

Table 5.1 – Side-by-side comparison of both approaches

Approach	RMSE (MAD)	MAE (MAD)	R^2
Random Forest + TF-IDF + OHE	388.77	155.88	0.8035
XGBoost + BGE-M3 + Target Encoding	388.83	155.31	0.8034

5.2 Discussion

Both approaches achieve virtually identical performance ($R^2 \approx 0.80$, $MAE \approx 155$ MAD). This convergence, despite substantially different feature representations and model architectures strongly suggests that the performance ceiling is determined by the *information content of the available features* rather than by the choice of encoding or model.

5.2.1 Why Performance Did Not Improve in the Second Approach

1. **Short text inputs:** the extracted fields (`name`, `usage`, `composition`) are typically short phrases of fewer than 15 words. For very short texts, TF-IDF and semantic embeddings capture similar information, as there is limited context for the embedding model to exploit.
2. **Inherent price noise:** the same product may be listed at different prices by different sellers, creating irreducible noise in the target variable that no feature representation can eliminate.
3. **Missing features:** critical price determinants are absent: seller rating, number of reviews, product condition (new vs. refurbished), stock availability, and promotional pricing.

5.2.2 Significance of $R^2 = 0.80$ in the Customs Context

An R^2 of 0.80 means the system can explain 80% of retail price variance from product metadata alone. In the customs valuation context, this translates to a useful screening tool: when the declared value of an imported product deviates significantly from the model's estimate, this constitutes a signal warranting closer inspection. The remaining 20% of unexplained variance reflects factors not captured in public listing data (seller strategy, market timing, promotional cycles), which is acceptable for a screening-level reference tool.

5.3 Limitations

- **Category imbalance:** *Maison & Bureau* contains only 170 products, limiting model accuracy for this category.
- **Missing contextual features:** seller-level and market-level data are not captured.
- **Static snapshot:** prices fluctuate over time; the model requires periodic retraining.
- **CPU-bound computation:** embedding 10,000 texts with BGE-M3 on CPU takes approximately one hour, limiting rapid experimentation.

Chapter 6

Conclusion and Future Work

6.1 Conclusion

This internship project successfully designed, implemented, and evaluated a complete end-to-end machine learning pipeline for product price prediction on Jumia Morocco. Starting from an empty dataset, the work progressed through five distinct phases: compliant data collection via web scraping, data cleaning and preprocessing, LLM-based information extraction, feature engineering in two configurations of increasing sophistication, and model training and evaluation.

Both experimental approaches: Random Forest with TF-IDF and One-Hot Encoding, and XGBoost with BGE-M3 embeddings and Mean Target Encoding, consistently achieved an $R^2 \approx 0.80$ and a MAE ≈ 155 MAD. The convergence of both approaches confirms that the performance ceiling is driven by the available feature information rather than model choice, and that an $R^2 = 0.80$ constitutes a strong result for a heterogeneous, real-world e-commerce pricing task.

From the perspective of the internship at ADII, this project establishes a proof of concept for an automated price reference system. Given product metadata, the system produces a market price estimate that can serve as a benchmark for customs officers evaluating the plausibility of declared values on imported consumer goods.

6.2 Future Work

- **Category-specific models:** training separate models per product category would allow each model to specialise in category-level pricing patterns, likely reducing MAE for individual categories.
- **Dataset enrichment:** scraping additional attributes (seller ratings, review counts, official seller status, stock levels) would provide richer predictive signals.
- **Advanced modelling:** fine-tuning a pre-trained French language model (e.g. CamemBERT) end-to-end as a regression model may capture more complex feature interactions than separate encoding and regression steps.
- **Temporal modelling:** incorporating price history and periodic retraining would adapt the system to market dynamics and seasonal price variations.
- **API deployment:** packaging the trained pipeline as a REST API would enable integration with ADII's information systems (BADR), allowing real-time price querying on product descriptions submitted at the border.